Algorítmica y Programación II | Grupo 2 | UNPSJB

Proyecto	Sistema de Gestión de Equivalencias de Materiales - Flota RASU
Materia	Algorítmica y Programación II
Grupo	2
Propósito	Guía de estructura y flujo de trabajo para la implementación de TADs
Lenguaje	Ada

1. Introducción

Este documento describe la arquitectura del sistema RASU: la organización de paquetes, las estructuras de datos elegidas para cada TAD, las operaciones que cada módulo debe implementar y el flujo de interacción entre ellos.

El objetivo es que cada uno pueda desarrollar su TAD de manera organizada, respetando las interfaces definidas en los archivos .ads, para que la integración final sea directa.

2. Visión general del sistema

El sistema gestiona la flota de aviones de la Fuerza Aérea Argentina (Región Sur) y sus insumos con equivalencias de otras marcas. Se estructura en cuatro TADs principales más un módulo de menú y submenús.

2.1 Módulos del sistema

```
Dependencias de paquetes (with/use)
Menu_Principal
  with Aviones, Repuestos, Equivalencias, Submenus

Submenus
  with Aviones, Repuestos, Equivalencias

Aviones      -- TAD independiente, usa ListaEnlazada
Repuestos    -- TAD independiente, usa ListaEnlazada
Equivalencias -- usa ListaEnlazada, referencia a Aviones y Repuestos

Paquetes genericos (provistos, no modificar):
  ListaEnlazada  -- lista dinámica genérica
  ColaDinamica   -- cola FIFO genérica
  NT_Console     -- colores y control de pantalla (Windows)
```

2.2 Flujo de ejecución

El programa arranca en Menu_Principal. Al iniciarse, intenta abrir los archivos .dat de cada TAD. Si no existen, los crea e inicializa con datos de prueba. Luego entra en el loop del menú.

Cada opción del menú llama a un submenú dentro de Submenus.adb, que a su vez invoca los procedimientos del TAD correspondiente.

Ciclo de vida del programa

1. Inicio: cargar datos desde archivos (.dat) a listas en memoria
2. Loop menú: recibir opción del usuario
3. Submenú: llamar al procedimiento del TAD (Alta/Baja/Modificar/Listar)
4. TAD: operar sobre la lista dinámica en memoria
5. Archivo: guardar cambios al .dat al salir o por operación

3. Diseño descendente

Árbol de módulos

- 1. Sistema RASU
  - 1.1. Aviones
    - 1.1.1. ABM (Alta, Baja, Modificar)
  - 1.2. Repuestos Originales
    - 1.2.1. ABM de repuestos
    - 1.2.2. ABM de equivalencias (lista anidada)
  - 1.3. Interfaz Menú
    - 1.3.1. Menú principal
    - 1.3.2. Submenús por TAD
  - 1.4. Reportes
    - 1.4.1. Insumo alternativo más usado (último año)
    - 1.4.2. Aviones y equivalencias por tipo
    - 1.4.3. Top 3 alternativas rentables por tipo
    - 1.4.4. Equivalencias nac/int por tipo de avión
    - 1.4.5. Alternativas por insumo y aviones asociados

4. TADs — Especificación e implementación

Cada TAD tiene su propio par de archivos: nombre.ads (especificación) y nombre.adb (implementación). Los .ads definen la interfaz pública; los .adb la implementan. Nunca se cambia la signatura pública sin avisar.

4.1 TAD Aviones

Archivo: aviones.ads / aviones.adb

Estructura base: Lista dinámica enlazada (ListaEnlazada genérico instanciado con RegAvion).

Registro principal

Campo	Tipo	Descripcion
Codigo	Positive	Identificador único del avión (clave de búsqueda)
Tipo	String(1..20)	Tipo de avión: Pampa, A-4, Super Etendard, etc.
LongTipo	Positive	Longitud real de la cadena Tipo
Modelo	String(1..20)	Modelo específico dentro del tipo
LongMod	Positive	Longitud real de la cadena Modelo
Anio	Positive	Año de incorporación a la flota

Operaciones requeridas

Procedimiento / Funcion	Descripcion
Alta(L, Avion)	Inserta un avión en la lista. Lanza Duplicado si el Codigo ya existe.
Baja(L, Codigo)	Elimina el avión con ese Codigo. Lanza No_Encontrado si no existe.
Modificar(L, Codigo, ...)	Busca por Codigo y actualiza los campos recibidos.
Buscar(L, Codigo)	Retorna el registro del avión. Lanza No_Encontrado si no está.
BuscarPorTipo(L, Tipo)	Retorna sublista con todos los aviones de ese tipo.
Listar(L)	Imprime todos los aviones por pantalla.
Guardar(L, Archivo)	Serializa la lista al archivo aviones.dat con Sequential_IO.
Cargar(L, Archivo)	Deserializa el archivo a la lista en memoria al iniciar.

Excepciones

Avion_Duplicado
Avion_No_Encontrado

4.2 TAD Repuestos

Archivo: repuestos.ads / repuestos.adb

Estructura base: Lista dinámica enlazada. Cada nodo contiene el repuesto original y una lista anidada de equivalencias.

Tipos

Campo	Tipo	Descripcion
TipoRepuesto	Tipo enumerado	Filtro_Aceite, Filtro_Aire, Correa, Bujia, Neumatico, ... (agregar según enunciado)
Origen	Tipo enumerado	Nacional / Internacional

Registro RepuestoOriginal

Campo	Tipo	Descripcion
Codigo	Positive	Identificador único del repuesto
Tipo	TipoRepuesto	Categoría del repuesto (enumerado)
Descripcion	String(1..50)	Nombre descriptivo, ej: Filtro de aceite de motor
LongDesc	Positive	Longitud real de la cadena
CodigoAvion	Positive	Avión al que pertenece este repuesto
Equivalencias	ListaEquiv	Lista anidada de alternativas (ver TAD Equivalencias)

Operaciones requeridas

Procedimiento / Funcion	Descripción
AltaRepuesto(L, Rep)	Inserta nuevo repuesto. Lanza Duplicado si el Codigo ya existe.
BajaRepuesto(L, Codigo)	Elimina repuesto y todas sus equivalencias.
ModRepuesto(L, Codigo, ...)	Modifica campos del repuesto original (no las equivalencias).
BuscarRepuesto(L, Codigo)	Retorna el nodo del repuesto para poder acceder a su lista de equiv.
ListarPorTipo(L, Tipo)	Lista todos los repuestos de un TipoRepuesto dado.
ListarPorAvion(L, CodAv)	Lista repuestos cuyo CodigoAvion coincide.
Guardar(L, Archivo)	Serializa la lista al archivo repuestos.dat.
Cargar(L, Archivo)	Carga el archivo a la lista en memoria.

4.3 TAD Equivalencias

Archivo: equivalencias.ads / equivalencias.adb

Estructura base: Lista dinámica enlazada, anidada dentro de cada nodo de Repuestos.

Registro RegEquivalencia

Campo	Tipo	Descripcion
CodigoEquiv	Positive	Identificador único de la equivalencia
Marca	String(1..30)	Marca del insumo alternativo
LongMarca	Positive	Longitud real de la cadena
Origen	Origen	Nacional o Internacional
Precio	Float	Precio actual del insumo
CantUsadaAnio	Natural	Cantidad utilizada en el último año (para reportes)
CodigoRepuesto	Positive	Repuesto original al que reemplaza
CodigoAvion	Positive	Avion al que está asociada

Operaciones requeridas

Procedimiento / Funcion	Descripcion
AltaEquiv(L, Eq)	Agrega una equivalencia a la lista del repuesto.
BajaEquiv(L, CodigoEquiv)	Elimina la equivalencia.
ModEquiv(L, CodigoEquiv, ...)	Modifica precio, marca u origen.
BuscarEquiv(L, CodigoEquiv)	Retorna el registro de equivalencia.
ListarEquiv(L)	Lista todas las equivalencias de un repuesto.
MasUsadaAnio(L)	Retorna la equivalencia con mayor CantUsadaAnio (reporte 1).
Top3Rentables(L)	Retorna los 3 repuestos con mejor relación precio/uso (reporte 3).
Guardar(L, Archivo)	Serializa al archivo equivalencias.dat.
Cargar(L, Archivo)	Carga el archivo.

5. Patrón de manejo de archivos

Todos los TADs siguen el mismo patrón de persistencia. Es importante que cada uno lo respete para que la integración funcione correctamente.

Patron común (tomado del TAD Cliente del trabajo anterior)

```
-- En el .ads:
package Archi_X is new Ada.Sequential_IO(RegX);

procedure Guardar(L: in TipoLista; Archivo: String);
procedure Cargar (L: out TipoLista; Archivo: String);

-- En el .adb (Guardar):
procedure Guardar(L: in TipoLista; Archivo: String) is
  Arch : Archi_X.File_Type;
  Ptr  : TipoLista := L;
begin
  Archi_X.Create(Arch, Archi_X.Out_File, Archivo);
  while not Vacia(Ptr) loop
    Archi_X.Write(Arch, Info(Ptr));
    Ptr := Sig(Ptr);
  end loop;
  Archi_X.Close(Arch);
end Guardar;

-- En el .adb (Cargar):
procedure Cargar(L: out TipoLista; Archivo: String) is
  Arch : Archi_X.File_Type;
  Elem : RegX;
begin
  Crear(L);
  Archi_X.Open(Arch, Archi_X.In_File, Archivo);
  while not Archi_X.End_Of_File(Arch) loop
    Archi_X.Read(Arch, Elem);
    Insertar(L, Elem);
  end loop;
  Archi_X.Close(Arch);
exception
  when Archi_X.Name_Error => null; -- archivo no existe, lista queda vacía
end Cargar;
```

Nota: el tipo serializado al archivo NO puede contener punteros (listas anidadas). Para las equivalencias anidadas se necesita un archivo separado, dos archivos para Reparaciones (reparaciones.dat y partesreparadas.dat). Ver sección 6.

6. Manejo de listas anidadas en archivo

El TAD Repuestos tiene una lista de equivalencias dentro de cada nodo. Ada no puede serializar punteros directamente con Sequential_IO. La solución es usar dos archivos:

- repuestos.dat — almacena los campos escalares de cada RepuestoOriginal (sin la lista de equiv)
- equivalencias.dat — almacena cada equivalencia con su CodigoRepuesto como clave foránea

Al cargar, primero se levanta repuestos.dat, luego equivalencias.dat: por cada equivalencia se busca el repuesto por CodigoRepuesto y se inserta en su lista interna.

```
Tipos para archivo (sin punteros)
type TArchiRepuesto is record
  Codigo      : Positive;
  Tipo        : TipoRepuesto;
  Descripcion : String(1..50);
  LongDesc    : Positive;
  CodigoAvion : Positive;
  -- sin Equivalencias (es lista dinámica)
end record;

type TArchiEquivalencia is record
  CodigoEquiv    : Positive;
  Marca          : String(1..30);
  LongMarca      : Positive;
  Origen         : Origen;
  Precio         : Float;
  CantUsadaAnio  : Natural;
  CodigoRepuesto : Positive;  -- clave foránea
  CodigoAvion    : Positive;
end record;
```

7. Submenús — Patrón de integración

El archivo submenus.adb es el punto de integración. Cada procedimiento de gestión recibe las listas como parámetros in out y llama a los procedimientos del TAD correspondiente. El patrón es el mismo para todos:

Patrón submenú (ejemplo Aviones)

```
procedure Gestion_Aviones(Aviones: in out TAviones) is
  Rta : Character;
begin
  loop
    Put_Line("| 1. Alta de avion                    |");
    Put_Line("| 2. Baja de avion                    |");
    Put_Line("| 3. Modificar avion                  |");
    Put_Line("| 4. Listar aviones                   |");
    Put_Line("| 5. Volver                           |");
    Get(Rta); Skip_Line;
    case Rta is
      when 1 => Alta_Avion(Aviones);
      when 2 => Baja_Avion(Aviones);
      when 3 => Modificar_Avion(Aviones);
      when 4 => Listar_Aviones(Aviones);
      when 5 => exit;
      when others => Put_Line("Opcion invalida.");
    end case;
    Skip_Line;
  end loop;
end Gestion_Aviones;
```

8. Convenciones de código

8.1 Nomenclatura

- Los tipos comienzan con T mayúscula: TAviones, TRepuestos, TEquivalencias.
- Los registros comienzan con Reg: RegAvion, RegRepuesto, RegEquivalencia.
- Los tipos para archivo (sin punteros) comienzan con TArchi: TArchiRepuesto.
- Las excepciones son descriptivas y terminan en la entidad: Avion_Duplicado, Repuesto_No_Encontrado.
- Los archivos .dat llevan el nombre en minúsculas: aviones.dat, repuestos.dat.

8.2 Manejo de cadenas

Ada requiere strings de longitud fija. El patrón es: String(1..N) para el campo + Positive para la longitud real. Al imprimir: Nombre(1..LongNombre). Al comparar: usar la longitud real para recortar.

Ejemplo de lectura y comparación de cadenas

```
-- Lectura:
Get_Line(Dato.Tipo, Dato.LongTipo);

-- Impresión:
Put(Dato.Tipo(1..Dato.LongTipo));

-- Comparación:
if Info(Ptr).Tipo(1..Info(Ptr).LongTipo) = Buscar(1..LongBuscar) then
```

8.3 Limpiar el buffer de entrada

Cada Get numérico deja el Enter en el buffer. Siempre hacer Skip_Line después de Get(Integer/Float/etc.) y antes del próximo Get_Line.

9. División de tareas sugerida

TAD Aviones	A definir
TAD Repuestos	A definir
TAD Equivalencias	A definir
Menú + Submenús	En grupo
Reportes	En grupo
Archivos + pruebas	En grupo
Programa Principal	En grupo

```
sistema_rasu.exe — NT_Console

|SISTEMA RASU v1.0                          |     |GESTION DE AVIONES
| 1. Gestion de Aviones |                         | 1. Alta de avion |
| 2. Gestion de Repuestos |                       | 2. Baja de avion |
| 3. Gestion de Equivalencias |                   | 3. Modificar avion |
| 4. Reportes |                            opcion 1 | 4. Listar aviones |
| 5. Salir |                                  ──→    | 5. Volver al menu principal |

Seleccione una opcion:   1                        Seleccione una opcion:   1

=== ALTA DE AVION ===                             ▫ Avion registrado exitosamente.
Codigo (ID unico):                101             Codigo:    101
Tipo (Pampa, A-4, Etendard..):     Pampa     OK   Tipo:      Pampa
Modelo:                  IA-63                ──→ Modelo:    IA-63
Anio de incorporacion:        1988               Anio:      1988
Confirmar alta? [s/n]: s                          -- Guardado en aviones.dat --

                                                  Si el código ya existe:
                                                  ERROR: Codigo duplicado. El avion 101
                                                  ya existe en el sistema.

=== REPORTE: EQUIVALENCIAS POR TIPO === Tipo:    Pampa
-----------------------------------------------
Cod Tipo avion Repuesto Alternativa
101   Pampa/IA-63 Filtro aceite            AcmeParts FO-78
101   Pampa/IA-63 Filtro aceite            RioFil 320-N
```

Colores NT_Console usados en el sistema

Set_Foreground(White) → bordes del menú y etiquetas principales
Set_Foreground(Gray) → opciones de menú y texto secundario
Set_Foreground(Cyan) → datos ingresados por el usuario
Set_Foreground(Green) → mensajes de éxito (alta OK, guardado)
Set_Foreground(Red) → errores (duplicado, no encontrado, inválido)
Set_Foreground(Yellow) → títulos de sección / nombre del sistema

Plantillas ADS:

```
-- PLANTILLA GENÉRICA DE TAD — SISTEMA RASU  (.ads)

with ListaEnlazada;
with Ada.Sequential_IO;

package XXXX is

  -- ==== 1. TIPOS ENUMERADOS (Solo si se necesitan) ====
  -- type TipoXXXX is (Opcion1, Opcion2, Opcion3);

  -- ==== 2. REGISTRO PRINCIPAL ====
  -- Cada campo de texto lleva su LongXXXX asociado.
  -- Ada usa strings de longitud fija: String(1..N) + Natural para la longitud real.
  -- Al imprimir se usa Campo(1..LongCampo); al comparar también.
  type RegXXXX is record
    Codigo     : Positive;        -- clave primaria (búsqueda/baja/modificar)
    Campo1     : String(1..50);    -- ajustar tamaño según el dato
    LongCampo1 : Natural;
    Campo2     : String(1..100);
    LongCampo2 : Natural;
    -- CampoNum  : Positive;       -- numérico: sin Long asociado
    -- CampoEnum : TipoXXXX;       -- enumerado: sin Long asociado
  end record;

  -- ==== 3. LISTA ====
  package Lista_XXXX is new ListaEnlazada(RegXXXX);
  use Lista_XXXX;

  subtype TXXXX is TipoLista;

  -- ==== 4. EXCEPCIONES ====
  XXXX_Duplicado     : exception;   -- se lanza en Alta si el Codigo ya existe
  XXXX_No_Encontrado : exception;   -- se lanza en Baja/Modificar/Buscar

  -- ==== 5. OPERACIONES ====
  -- La lista siempre entra como parámetro (nunca variable global).
  -- "in out" cuando la operación modifica la lista, "in" cuando solo consulta.

  procedure Alta     (L: in out TXXXX; Reg: in RegXXXX);
  procedure Baja     (L: in out TXXXX; Codigo: in Positive);
  procedure Modificar(L: in out TXXXX; Codigo: in Positive; NuevoReg: in RegXXXX);
  function  Buscar   (L: in TXXXX;     Codigo: in Positive) return RegXXXX;
  procedure Listar   (L: in TXXXX);

  -- ==== 6. PERSISTENCIA ====
  -- Sequential_IO instanciado con el registro. Como RegXXXX no tiene punteros,
  -- se serializa directo — no necesitamos un tipo TArchi separado.
  package ArchiXXXX is new Ada.Sequential_IO(RegXXXX);

  procedure Guardar(L: in TXXXX;  Archivo: in String);
  procedure Cargar (L: out TXXXX; Archivo: in String);

end XXXX;
```

Plantilla ADB:

```
-- ==============================================================================
-- PLANTILLA GENÉRICA DE TAD — SISTEMA RASU  (.adb)
-- ==============================================================================

with Ada.Text_IO;        use Ada.Text_IO;
with Ada.Integer_Text_IO; use Ada.Integer_Text_IO;
with NT_Console;         use NT_Console;

package body XXXX is

  -- ==== AUXILIAR INTERNO: BuscarPtr ====
  -- No va en el .ads porque es un detalle de implementación.
  -- Recibe una copia de la lista para recorrerla sin modificar L.
  -- Si lo encuentra, Enc=True y Ptr queda apuntando al nodo.
  procedure BuscarPtr(Ptr: in out TXXXX; Codigo: in Positive; Enc: out Boolean) is
  begin
    Enc := False;
    while not Vacia(Ptr) and then not Enc loop
      if Info(Ptr).Codigo = Codigo then
        Enc := True;
      else
        Ptr := Sig(Ptr);
      end if;
    end loop;
  end BuscarPtr;


  -- ==== ALTA ====
  procedure Alta(L: in out TXXXX; Reg: in RegXXXX) is
    Ptr : TXXXX := L;
    Enc : Boolean;
  begin
    BuscarPtr(Ptr, Reg.Codigo, Enc);
    if Enc then
      raise XXXX_Duplicado;
    end if;
    InsertarPorFinal(L, Reg);
  end Alta;


  -- ==== BAJA ====
  procedure Baja(L: in out TXXXX; Codigo: in Positive) is
    Ptr : TXXXX := L;
    Enc : Boolean;
  begin
    BuscarPtr(Ptr, Codigo, Enc);
    if not Enc then
      raise XXXX_No_Encontrado;
    end if;
    Suprimir(L, Info(Ptr));
  end Baja;


  -- ==== MODIFICAR ====
  -- Se suprime el nodo viejo y se inserta unol nuevo al final.
  procedure Modificar(L: in out TXXXX; Codigo: in Positive; NuevoReg: in RegXXXX) is
    Ptr   : TXXXX := L;
    Enc   : Boolean;
    Viejo : RegXXXX;
  begin
    BuscarPtr(Ptr, Codigo, Enc);
    if not Enc then
      raise XXXX_No_Encontrado;
    end if;
    Viejo := Info(Ptr);
    Suprimir(L, Viejo);
    InsertarPorFinal(L, NuevoReg);
  end Modificar;
```

```
  -- ==== BUSCAR ====
  function Buscar(L: in TXXXX; Codigo: in Positive) return RegXXXX is
    Ptr : TXXXX := L;
    Enc : Boolean;
  begin
    BuscarPtr(Ptr, Codigo, Enc);
    if not Enc then
      raise XXXX_No_Encontrado;
    end if;
    return Info(Ptr);
  end Buscar;


  -- ==== LISTAR ====
  -- Campo(1..LongCampo) imprime solo los caracteres reales, sin espacios de más (osea no imprime basura)
  procedure Listar(L: in TXXXX) is
    Ptr : TXXXX := L;
  begin
    if Vacia(Ptr) then
      Set_Foreground(Red);
      Put_Line("No hay registros cargados.");
      Set_Foreground(White);
      return;
    end if;
    Set_Foreground(Yellow);
    Put_Line("========== LISTADO DE XXXX ==========");
    Set_Foreground(White);
    while not Vacia(Ptr) loop
      Put("Codigo : "); Put(Info(Ptr).Codigo, 1); New_Line;
      Put("Campo1 : "); Put(Info(Ptr).Campo1(1..Info(Ptr).LongCampo1)); New_Line;
      Put("Campo2 : "); Put(Info(Ptr).Campo2(1..Info(Ptr).LongCampo2)); New_Line;
      Put_Line("----------------------------------------");
      Ptr := Sig(Ptr);
    end loop;
  end Listar;


  -- ==== GUARDAR ====
  -- Create sobreescribe si el archivo ya existe
  procedure Guardar(L: in TXXXX; Archivo: in String) is
    Arch : ArchiXXXX.File_Type;
    Ptr  : TXXXX := L;
  begin
    ArchiXXXX.Create(Arch, ArchiXXXX.Out_File, Archivo);
    while not Vacia(Ptr) loop
      ArchiXXXX.Write(Arch, Info(Ptr));
      Ptr := Sig(Ptr);
    end loop;
    ArchiXXXX.Close(Arch);
  end Guardar;


  -- ==== CARGAR ====
  -- Name_Error silenciado: primera ejecución sin archivo → lista vacía, no es error.
  procedure Cargar(L: out TXXXX; Archivo: in String) is
    Arch : ArchiXXXX.File_Type;
    Elem : RegXXXX;
  begin
    Crear(L);
    ArchiXXXX.Open(Arch, ArchiXXXX.In_File, Archivo);
    while not ArchiXXXX.End_Of_File(Arch) loop
      ArchiXXXX.Read(Arch, Elem);
      InsertarPorFinal(L, Elem);
    end loop;
    ArchiXXXX.Close(Arch);
  exception
    when ArchiXXXX.Name_Error => null;
  end Cargar;

end XXXX;
```

Plantilla submenú

```
-- ==============================================================================
-- PLANTILLA DE SUBMENÚ — SISTEMA RASU
-- Va dentro de submenus.adb.
-- ==============================================================================

  function LeerXXXX(L: in TXXXX) return RegXXXX is
    Reg : RegXXXX;
  begin
    -- Código numérico con validación de tipo
    loop
      begin
        Put("Codigo: ");
        Get(Reg.Codigo);
        Skip_Line;
        exit;
      exception
        when Ada.IO_Exceptions.Data_Error =>
          Skip_Line;
          Set_Foreground(Red);
          Put_Line("Valor invalido. Intente de nuevo.");
          Set_Foreground(White);
      end;
    end loop;

    Put("Campo1: ");
    Get_Line(Reg.Campo1, Reg.LongCampo1);

    Put("Campo2: ");
    Get_Line(Reg.Campo2, Reg.LongCampo2);

    return Reg;
  end LeerXXXX;


  -- ==== SUBMENÚ PRINCIPAL ====
  -- La lista entra siempre como in out — nunca variable global.
  -- El loop corre hasta que el usuario elige Volver.
  procedure Gestion_XXXX(L: in out TXXXX) is
    Rta : Character;
  begin
    loop
      Set_Foreground(White);
      Put_Line("+----------------------------------------------+");
      Set_Foreground(Yellow);
      Put_Line("|            GESTION DE XXXX                   |");
      Set_Foreground(White);
      Put_Line("+----------------------------------------------+");
      Set_Foreground(Gray);
      Put_Line("| 1. Alta                                      |");
      Put_Line("| 2. Baja                                      |");
      Put_Line("| 3. Modificar                                 |");
      Put_Line("| 4. Listar                                    |");
      Put_Line("| 5. Volver al menu principal                  |");
      Put_Line("+----------------------------------------------+");
      Set_Foreground(White);
      New_Line;
```

```
Put("Seleccione una opcion: ");
Get(Rta);
Skip_Line;
New_Line;

case Rta is

  when '1' =>   -- ALTA
    declare
      Reg : RegXXXX;
    begin
      Set_Foreground(Yellow);
      Put_Line("=== ALTA DE XXXX ===");
      Set_Foreground(White);
      Reg := LeerXXXX(L);
      Alta(L, Reg);
      Set_Foreground(Green);
      Put_Line("Registrado exitosamente.");
      Set_Foreground(White);
    exception
      when XXXX_Duplicado =>
        Set_Foreground(Red);
        Put_Line("ERROR: El codigo ya existe.");
        Set_Foreground(White);
    end;

  when '2' =>   -- BAJA
    declare
      Codigo : Positive;
    begin
      Set_Foreground(Yellow);
      Put_Line("=== BAJA DE XXXX ===");
      Set_Foreground(White);
      Listar(L);
      New_Line;
      Put("Codigo a eliminar: ");
      Get(Codigo);
      Skip_Line;
      Baja(L, Codigo);
      Set_Foreground(Green);
      Put_Line("Eliminado correctamente.");
      Set_Foreground(White);
    exception
      when XXXX_No_Encontrado =>
        Set_Foreground(Red);
        Put_Line("ERROR: Codigo no encontrado.");
        Set_Foreground(White);
      when Ada.IO_Exceptions.Data_Error =>
        Skip_Line;
        Set_Foreground(Red);
        Put_Line("ERROR: Codigo invalido.");
        Set_Foreground(White);
    end;

  when '3' =>   -- MODIFICAR
    declare
      Codigo   : Positive;
      NuevoReg : RegXXXX;
      Actual   : RegXXXX;
```

```
          begin
            Set_Foreground(Yellow);
            Put_Line("=== MODIFICAR XXXX ===");
            Set_Foreground(White);
            Listar(L);
            New_Line;
            Put("Codigo a modificar: ");
            Get(Codigo);
            Skip_Line;
            Actual := Buscar(L, Codigo);
            Put("  Campo1 actual: "); Put(Actual.Campo1(1..Actual.LongCampo1)); New_Line;
            Put("  Campo2 actual: "); Put(Actual.Campo2(1..Actual.LongCampo2)); New_Line;
            New_Line;
            Put_Line("Ingrese los nuevos datos:");
            NuevoReg        := LeerXXXX(L);
            NuevoReg.Codigo := Codigo;   -- conservar codigo original
            Modificar(L, Codigo, NuevoReg);
            Set_Foreground(Green);
            Put_Line("Modificado correctamente.");
            Set_Foreground(White);
          exception
            when XXXX_No_Encontrado =>
              Set_Foreground(Red);
              Put_Line("ERROR: Codigo no encontrado.");
              Set_Foreground(White);
            when Ada.IO_Exceptions.Data_Error =>
              Skip_Line;
              Set_Foreground(Red);
              Put_Line("ERROR: Codigo invalido.");
              Set_Foreground(White);
          end;

        when '4' =>   -- LISTAR
          Listar(L);

        when '5' =>   -- VOLVER
          Clear_Screen;
          exit;

        when others =>
          Set_Foreground(Red);
          Put_Line("Opcion invalida.");
          Set_Foreground(White);

      end case;

      New_Line;
      Put_Line("Presione ENTER para continuar...");
      Skip_Line;
      Clear_Screen;

    end loop;
  end Gestion_XXXX;
```

Repuestos ADS:

```
-- ==============================================================================
-- TAD REPUESTOS — SISTEMA RASU  (.ads)
-- En este TAD cada nodo tiene una lista anidada de Equivalencias.
-- Por eso se usan dos archivos separados.
-- ==============================================================================

with ListaEnlazada;
with Ada.Sequential_IO;
with Equivalencias; use Equivalencias;

package Repuestos is

  -- ==== 1. TIPOS ====
  type TipoRepuesto is (Filtro_Aceite, Filtro_Aire, Correa, Bujia, Neumatico);
  -- Agregar según enunciado


  -- ==== 2. REGISTRO EN MEMORIA ====
  -- El campo Equivalencias es una lista dinámica (contiene punteros).
  -- Por eso no puede ir directo a Sequential_IO → ver sección 6.
  type RegRepuesto is record
    Codigo       : Positive;
    Tipo         : TipoRepuesto;
    Descripcion   : String(1..50);
    LongDesc      : Natural;
    CodigoAvion   : Positive;
    Equivalencias : TEquivalencias;   -- lista anidada en memoria
  end record;


  -- ==== 3. REGISTRO PARA ARCHIVO repuestos.dat ====
  -- Mismo registro pero SIN el campo Equivalencias.
  -- Las equivalencias se guardan por separado en equivalencias.dat
  -- usando CodigoRepuesto como clave foránea.
  type TArchiRepuesto is record
    Codigo      : Positive;
    Tipo        : TipoRepuesto;
    Descripcion : String(1..50);
    LongDesc    : Natural;
    CodigoAvion : Positive;
  end record;


  -- ==== 4. LISTA ====
  package Lista_Repuestos is new ListaEnlazada(RegRepuesto);
  use Lista_Repuestos;

  subtype TRepuestos is TipoLista;


  -- ==== 5. EXCEPCIONES ====
  Repuesto_Duplicado     : exception;
  Repuesto_No_Encontrado : exception;


  -- ==== 6. OPERACIONES ====
```

```
   procedure Alta     (L: in out TRepuestos; Reg: in RegRepuesto);
   procedure Baja     (L: in out TRepuestos; Codigo: in Positive);
   procedure Modificar(L: in out TRepuestos; Codigo: in Positive; NuevoReg: in RegRepuesto);
   function  Buscar   (L: in TRepuestos;     Codigo: in Positive) return RegRepuesto;
   procedure Listar   (L: in TRepuestos);


   -- ==== 7. PERSISTENCIA — DOS ARCHIVOS ====
   -- Guardar:
   --   1. Escribe cada TArchiRepuesto (sin equiv) en repuestos.dat
   --   2. Escribe todas las equivalencias de todos los nodos en equivalencias.dat
   --      con CodigoRepuesto como clave (lo hace el TAD Equivalencias)
   -- Cargar:
   --   1. Lee repuestos.dat → reconstruye la lista con Equivalencias vacías
   --   2. Lee equivalencias.dat → por cada registro busca su repuesto
   --      por CodigoRepuesto y lo inserta en la lista interna
   --   IMPORTANTE: siempre se carga repuestos antes que equivalencias.
   package ArchiRepuesto is new Ada.Sequential_IO(TArchiRepuesto);

   procedure Guardar(L: in TRepuestos;  ArchivoRep: in String; ArchivoEquiv: in String);
   procedure Cargar (L: out TRepuestos; ArchivoRep: in String; ArchivoEquiv: in String);

end Repuestos;
```

Respuestos ADB:

```
-- ==============================================================================
-- TAD REPUESTOS — SISTEMA RASU  (.adb)
-- ==============================================================================

with Ada.Text_IO;        use Ada.Text_IO;
with Ada.Integer_Text_IO; use Ada.Integer_Text_IO;
with NT_Console;         use NT_Console;
with Equivalencias;      use Equivalencias;

package body Repuestos is

  -- ==== AUXILIAR INTERNO: BuscarPtr ====
  procedure BuscarPtr(Ptr: in out TRepuestos; Codigo: in Positive; Enc: out Boolean) is
  begin
    Enc := False;
    while not Vacia(Ptr) and then not Enc loop
      if Info(Ptr).Codigo = Codigo then
        Enc := True;
      else
        Ptr := Sig(Ptr);
      end if;
    end loop;
  end BuscarPtr;


  -- ==== ALTA ====
  procedure Alta(L: in out TRepuestos; Reg: in RegRepuesto) is
    Ptr : TRepuestos := L;
    Enc : Boolean;
  begin
    BuscarPtr(Ptr, Reg.Codigo, Enc);
    if Enc then raise Repuesto_Duplicado; end if;
    InsertarPorFinal(L, Reg);
  end Alta;


  -- ==== BAJA ====
  -- Elimina el repuesto y todas sus equivalencias (están dentro del nodo).
  procedure Baja(L: in out TRepuestos; Codigo: in Positive) is
    Ptr : TRepuestos := L;
    Enc : Boolean;
  begin
    BuscarPtr(Ptr, Codigo, Enc);
    if not Enc then raise Repuesto_No_Encontrado; end if;
    Suprimir(L, Info(Ptr));
  end Baja;


  -- ==== MODIFICAR ====
  -- Solo modifica los campos del repuesto, no toca la lista de equivalencias.
  procedure Modificar(L: in out TRepuestos; Codigo: in Positive; NuevoReg: in RegRepuesto) is
    Ptr   : TRepuestos := L;
    Enc   : Boolean;
    Viejo : RegRepuesto;
    Nuevo : RegRepuesto := NuevoReg;
  begin
    BuscarPtr(Ptr, Codigo, Enc);
```

```
   if not Enc then raise Repuesto_No_Encontrado; end if;
   Viejo := Info(Ptr);
   Nuevo.Equivalencias := Viejo.Equivalencias;   -- conservar equivalencias
   Suprimir(L, Viejo);
   InsertarPorFinal(L, Nuevo);
end Modificar;


-- ==== BUSCAR ====
function Buscar(L: in TRepuestos; Codigo: in Positive) return RegRepuesto is
   Ptr : TRepuestos := L;
   Enc : Boolean;
begin
   BuscarPtr(Ptr, Codigo, Enc);
   if not Enc then raise Repuesto_No_Encontrado; end if;
   return Info(Ptr);
end Buscar;


-- ==== LISTAR ====
procedure Listar(L: in TRepuestos) is
   Ptr : TRepuestos := L;
begin
   if Vacia(Ptr) then
      Set_Foreground(Red);
      Put_Line("No hay repuestos cargados.");
      Set_Foreground(White);
      return;
   end if;
   Set_Foreground(Yellow);
   Put_Line("========== LISTADO DE REPUESTOS ==========");
   Set_Foreground(White);
   while not Vacia(Ptr) loop
      Put("Codigo      : "); Put(Info(Ptr).Codigo, 1); New_Line;
      Put("Tipo        : "); Put_Line(TipoRepuesto'Image(Info(Ptr).Tipo));
      Put("Descripcion : "); Put(Info(Ptr).Descripcion(1..Info(Ptr).LongDesc)); New_Line;
      Put("Cod. Avion  : "); Put(Info(Ptr).CodigoAvion, 1); New_Line;
      Put_Line("----------------------------------------");
      Ptr := Sig(Ptr);
   end loop;
end Listar;


-- ==== GUARDAR ====
-- Pasada 1: escribe cada repuesto (sin equiv) en repuestos.dat
-- Pasada 2: por cada nodo llama a GuardarEquivalencias del TAD Equivalencias,
--           que escribe en equivalencias.dat con CodigoRepuesto como clave foránea.
procedure Guardar(L: in TRepuestos; ArchivoRep: in String; ArchivoEquiv: in String) is
   ArchRep  : ArchiRepuesto.File_Type;
   Ptr      : TRepuestos := L;
   Primera  : Boolean := True;
   RegArchi : TArchiRepuesto;
begin
   -- Pasada 1: repuestos.dat
   ArchiRepuesto.Create(ArchRep, ArchiRepuesto.Out_File, ArchivoRep);
   while not Vacia(Ptr) loop
      RegArchi.Codigo      := Info(Ptr).Codigo;
      RegArchi.Tipo        := Info(Ptr).Tipo;
      RegArchi.Descripcion := Info(Ptr).Descripcion;
```

```
      RegArchi.LongDesc    := Info(Ptr).LongDesc;
      RegArchi.CodigoAvion := Info(Ptr).CodigoAvion;
      ArchiRepuesto.Write(ArchRep, RegArchi);
      Ptr := Sig(Ptr);
    end loop;
    ArchiRepuesto.Close(ArchRep);

    -- Pasada 2: equivalencias.dat
    -- Primera=True indica que hay que crear el archivo; False que hay que hacer append.
    Ptr := L;
    while not Vacia(Ptr) loop
      GuardarEquivalencias(Info(Ptr).Equivalencias,
                           Info(Ptr).Codigo,
                           ArchivoEquiv,
                           Primera);
      Primera := False;
      Ptr := Sig(Ptr);
    end loop;
  end Guardar;


  -- ==== CARGAR ====
  -- Pasada 1: lee repuestos.dat → reconstruye L con listas de equiv vacías
  -- Pasada 2: lee equivalencias.dat → inserta cada equiv en su repuesto
  procedure Cargar(L: out TRepuestos; ArchivoRep: in String; ArchivoEquiv: in String) is
    ArchRep   : ArchiRepuesto.File_Type;
    ElemRep   : TArchiRepuesto;
    RegMem    : RegRepuesto;

    ArchEquiv : ArchiEquivalencia.File_Type;   -- declarado en TAD Equivalencias
    ElemEquiv : TArchiEquivalencia;
    PtrRep    : TRepuestos;
    Enc       : Boolean;
    Nodo      : RegRepuesto;
  begin
    -- Pasada 1: cargar repuestos
    Crear(L);
    ArchiRepuesto.Open(ArchRep, ArchiRepuesto.In_File, ArchivoRep);
    while not ArchiRepuesto.End_Of_File(ArchRep) loop
      ArchiRepuesto.Read(ArchRep, ElemRep);
      RegMem.Codigo      := ElemRep.Codigo;
      RegMem.Tipo        := ElemRep.Tipo;
      RegMem.Descripcion := ElemRep.Descripcion;
      RegMem.LongDesc    := ElemRep.LongDesc;
      RegMem.CodigoAvion := ElemRep.CodigoAvion;
      Crear(RegMem.Equivalencias);   -- lista interna vacía; se llena en pasada 2
      InsertarPorFinal(L, RegMem);
    end loop;
    ArchiRepuesto.Close(ArchRep);

    -- Pasada 2: cargar equivalencias y conectarlas a su repuesto
    ArchiEquivalencia.Open(ArchEquiv, ArchiEquivalencia.In_File, ArchivoEquiv);
    while not ArchiEquivalencia.End_Of_File(ArchEquiv) loop
      ArchiEquivalencia.Read(ArchEquiv, ElemEquiv);
      PtrRep := L;
      BuscarPtr(PtrRep, ElemEquiv.CodigoRepuesto, Enc);
      if Enc then
        Nodo := Info(PtrRep);
        InsertarEquiv(Nodo.Equivalencias, De_ArchiEquiv(ElemEquiv));
```

```
        Suprimir(L, Info(PtrRep));
        InsertarPorFinal(L, Nodo);
      end if;
    end loop;
    ArchiEquivalencia.Close(ArchEquiv);

  exception
    when ArchiRepuesto.Name_Error    => null;
    when ArchiEquivalencia.Name_Error => null;
  end Cargar;

end Repuestos;
```